# CS214 Coding Assignment HW0

Hugo Wan

Deadline: 4/13/2026

## 1. Machine Information

I used the `lscpu` command on the course server to obtain the hardware information of the tested machine. From the output, the machine has 48 logical CPUs in total, with 2 threads per core, 12 cores per socket, and 2 sockets. Therefore, the machine has 24 physical cores and 48 hardware threads in total. The cache information shown by `lscpu` reports L1d cache as 32KB, L1i cache as 32KB, L2 cache as 1024KB, and L3 cache as 16896KB.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ lscpu
Architecture:        x86_64
CPU op-mode(s):      32-bit, 64-bit
Byte Order:          Little Endian
CPU(s):              48
On-line CPU(s) list: 0-47
Thread(s) per core:  2
Core(s) per socket:  12
Socket(s):           2
NUMA node(s):        2
Vendor ID:           GenuineIntel
CPU family:          6
Model:               85
Model name:          Intel(R) Xeon(R) Silver 4214R CPU @ 2.40GHz
Stepping:            7
CPU MHz:             3500.000
CPU max MHz:         3500.0000
CPU min MHz:         1000.0000
BogoMIPS:            4800.00
Virtualization:      VT-x
L1d cache:           32K
L1i cache:           32K
L2 cache:            1024K
L3 cache:            16896K
NUMA node0 CPU(s):   0-11,24-35
NUMA node1 CPU(s):   12-23,36-47
Flags:               fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pa
t pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm
constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aper
fmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr p
dcm pcid dca sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c
rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l3 cdp_l3 invpcid_single intel_p
pin ssbd mba ibrs ibpb stibp ibrs_enhanced tpr_shadow vnmi flexpriority ept vpid ept_
ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid cqm mpx rdt_a avx512f avx512d
q rdseed adx smap clflushopt clwb intel_pt avx512cd avx512bw avx512vl xsaveopt xsavec
 xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local dtherm ida arat pln
 pts pku ospke avx512_vnni md_clear flush_l1d arch_capabilities
```

Figure 1: Output of `lscpu` used to identify the number of cores, hardware threads, and cache sizes on the tested machine.

## 2. Testing the Reduce Code with $n = 10^9$

The assignment asks to measure the runtime of reducing $10^9$ elements. I first implemented a fully sequential version that adds all elements one by one using a simple loop. Then I compared it against the provided parallel reduce implementation running with different thread counts. The assignment specifically asks for the sequential runtime and also for the runtime of the parallel algorithm using 1, 2, 4, 8, 12, 24, and 48 threads.

### 2(a) Sequential Runtime and Comparison with Parallel on One Core

The fully sequential version computes the sum using a simple loop. Its average runtime was approximately 1.14684 seconds. In comparison, the original parallel implementation running with one thread took 42.6831 seconds on average. This means the original parallel recursive version is much slower on one thread because it still pays the overhead of recursive task creation and synchronization, even though no real parallel speedup is being used.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 1.14112
Round 1 running time: 1.15114
Round 2 running time: 1.1437
Round 3 running time: 1.14568
Average running time: 1.14684
```

Figure 2: Sequential reduce runtime for $n = 10^9$. The average runtime was 1.14684 seconds.

### 2(b) Runtime with Different Thread Counts

I tested the original parallel reduce implementation using 1, 2, 4, 8, 12, 24, and 48 threads by changing the environment variable `PARLAY_NUM_THREADS`, as required in the assignment.

| Threads | Average Runtime (s) |
|---|---|
| 1 | 42.6831 |
| 2 | 21.5365 |
| 4 | 10.9888 |
| 8 | 5.70453 |
| 12 | 3.92238 |
| 24 | 2.09146 |
| 48 | 1.14230 |

Table 1: Average runtime of the original parallel reduce implementation with different thread counts for $n = 10^9$.

As the number of threads increases, the runtime decreases significantly. This shows that the reduce implementation benefits from parallelism, although the performance with one

thread is poor because the recursive parallel structure still introduces overhead. The best result among these tests was obtained with 48 threads, which achieved an average runtime of 1.14230 seconds.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=1
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 42.7305
Round 1 running time: 42.7302
Round 2 running time: 42.667
Round 3 running time: 42.6521
Average running time: 42.6831
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=2
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 21.6311
Round 1 running time: 21.5178
Round 2 running time: 21.529
Round 3 running time: 21.5626
Average running time: 21.5365
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=4
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 11.1211
Round 1 running time: 10.9018
Round 2 running time: 10.9757
Round 3 running time: 11.0889
Average running time: 10.9888
```

Figure 3: Program outputs for 1, 2, and 4 threads.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=8
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 5.82637
Round 1 running time: 5.73787
Round 2 running time: 5.70975
Round 3 running time: 5.66596
Average running time: 5.70453
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=12
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 3.92209
Round 1 running time: 3.95018
Round 2 running time: 3.91918
Round 3 running time: 3.89779
Average running time: 3.92238
```

Figure 4: Program outputs for 8 and 12 threads.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=24
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 2.08048
Round 1 running time: 2.10925
Round 2 running time: 2.07853
Round 3 running time: 2.08661
Average running time: 2.09146
[twan012@xe-15 hw0-reduce-HugoWan0504]$ export PARLAY_NUM_THREADS=48
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 1.14243
Round 1 running time: 1.14269
Round 2 running time: 1.14282
Round 3 running time: 1.14139
Average running time: 1.1423
```

Figure 5: Program outputs for 24 and 48 threads.

# 3. Granularity Control

The assignment explains that the original recursive reduce code may have unsatisfactory performance because scheduling and synchronization overhead become significant when the problem size becomes too small. To address this, I added granularity control so that when the input size is smaller than or equal to a threshold, the algorithm stops making recursive parallel calls and instead sums the elements directly using a sequential loop.

## 3(a) Algorithm with Granularity Control

I modified the recursive reduce function by adding a threshold-based base case. If the size $n$ is 0, the function returns 0. If $n$ is 1, it returns the single element. If $n$ is less than or equal to the threshold, it performs a sequential for-loop to sum the elements directly. Otherwise, it splits the array into two halves, recursively computes the partial sums in parallel using `par_do`, and returns the sum of the two results.

This change reduces the overhead caused by creating too many very small parallel tasks. Large subproblems still benefit from parallelism, while small subproblems are handled more efficiently by sequential execution.

## 3(b) Methodology for Finding a Good Threshold

To find a good threshold using only a small number of tests, I used a coarse-to-fine strategy. First, I tested thresholds across a broad range, including both small and large values, in order to identify the general region where performance was best. After that, I tested additional values near the better-performing region to refine the choice. This approach avoids exhaustive testing while still finding a near-optimal threshold.

## 3(c) Performance with Different Granularity Thresholds

I tested the performance of the modified reduce implementation using several threshold values. The results are shown below.

| Threshold $N$ | Average Runtime (s) |
|---|---|
| 100 | 0.119019 |
| 800 | 0.118151 |
| 1000 | 0.117262 |
| 1200 | 0.120331 |
| 4800 | 0.123784 |
| 10000 | 0.121157 |
| 90000 | 0.126544 |
| 100000 | 0.116997 |
| 120000 | 0.119497 |
| 1000000 | 0.122090 |

Table 2: Average runtime of the reduce implementation with different granularity-control thresholds for $n = 10^9$.

These results show that the threshold has a noticeable effect on performance. If the threshold is too small, the algorithm still creates too many small parallel tasks and suffers from overhead. If the threshold is too large, too much work is done sequentially, which reduces the benefit of parallelism. Therefore, the best threshold is a balance between these two effects.

## 3(d) Best Threshold and Runtime

Among the tested values, the best threshold was $N = 100000$, which achieved an average runtime of 0.116997 seconds. This was the lowest runtime in my experiments and was substantially faster than the original implementation without granularity control.

```
[twan012@xe-15 hw0-reduce-HugoWan0504]$ ./reduce 1000000000 3
Total sum: 499999999500000000
Warmup round running time: 0.119247
Round 1 running time: 0.115993
Round 2 running time: 0.119752
Round 3 running time: 0.115245
Average running time: 0.116997
```

Figure 6: Program output for the granularity-controlled reduce implementation with threshold $N = 100000$. The average runtime was 0.116997 seconds.

Overall, granularity control improved the performance significantly by reducing scheduling overhead for small subproblems. The final optimized runtime of about 0.117 seconds is also better than the assignment's target of about 0.3 seconds.

# References and Acknowledgements

I referred to the assignment handout and the provided code template while completing this homework. I also used ChatGPT to assist with understanding the questions, organizing the experiments, fixing grammar issues, and refining written explanations.
LaTeX template: `https://www.overleaf.com/read/ccjwvvjdfhzg#9e0f2b`
ChatGPT usage: `https://chatgpt.com/share/69dc593e-bfd8-8328-b691-c583709f94dc`